Hernández Nieves Emily

CORRECCION DE EXÁMEN

1. ¿Qué significa que un objeto sea una instancia de una clase? Explica si dos instancias deben tener iguales valores de sus atributos.

Una clase es como un molde que define las características y los comportamientos de los objetos. Una instancia es un ejemplo concreto que se crea a partir de ese molde. Dos instancias de una misma clase no tienen que tener los mismos valores en sus atributos, porque cada objeto puede guardar sus propios datos.

```
Class Perro {
   String nombre;
}
Perro p1 = new Perro ();
Perro p2 = new Perro();

p1.nombre = "Max";    // Aquí p1 y p2 son instancias de la clase perro,
p2.nombre = "Luna";   // pero tienen nombres diferentes.
```

2. ¿Qué diferencia hay entre un parametro y un argumento de un metodo? Ilustra con una llamada sencilla.

Un parametro es la variable que se declara en la cabecera de un metodo y sirve para recibir un dato. En cambio, un argumento es el valor que se proporciona cuando se llama al método

```
static int cargar (int cantidad) {
   return cantidad;
}
cargar (3);   // Cantidad es el parámetro y 3 el argumento que llama al
              // método.
```

3. ¿Qué es el ámbito de una variable local? Da un ejemplo de dónde se puede usar y donde deja de estar disponible.

El ámbito de una variable local es la parte del código en la que esa variable puede utilizarse. Generalmente una variable local se declara dentro de un método o bloque y solo puede usarse dentro de ese ámbito. Cuando se sale de ese bloque, ya no se puede acceder a ella.

```
static void ejemplo() {
    int numero = 5;                    // La variable numero puede utilizarse
    System.out.println(numero);        // dentro del método ejemplo. Fuera de
}                                      // este, Java marcará un error.
```

4. ¿Que guarda una variable de tipo arreglo? Explica que ocurre al asignarla a otra variable del mismo tipo.

Guarda una referencia al arreglo, no una copia independiente de todos sus elementos. Si se asigna la variable a otra del mismo tipo, ambas referencias apuntarán al mismo arreglo. Por eso, si se modifica un elemento usando cualquiera de las dos variables, el cambio también será observable al acceder al arreglo desde la otra.

5. ¿Que hace un break dentro de un case de un switch clásico? Explica qué puede ocurrir si falta.

la instruccion break hace que la ejecución salga del switch y continúe con la primera instruccion que este después de él. En caso de faltar un break, Java puede continuar ejecutando las instrucciones del siguiente case, aunque ese caso no coincida con el valor evaluado.

6. R1. ¿Qué imprime?

Salida:

4

5

6

E71

6.0

Tabla:

Vuelta	original + 2	original / 2	vuelta al terminar
1	9	4	1
2	11	5	2
3	13	6	3

- la division es entera porque original es en integer.
- la variable texto recibe el resultado de concatenar "E", el valor de copia y el 1. copia = 7. la concanetacion queda "E71".

7. R2 a) ¿Compila, truena o funciona?

```
String codigo = null;
boolean igual = codigo.equals("k3");
Sistem.out.println(igual);
```

Respuesta: Compila pero truena.
o la primera linea es valida, Java permite que una variable de tipo string tenga un valor null. El problema ocurre al ejecutar codigo.equals("k3"), porque codigo no hace referencia a ningun objeto. Al intentar llamar un metodo mediante una referencia null, se produce Nuil Pointer Exception.

8. R2 b) ¿Compila, truena o funciona?

```
double voltaje = 7.6;
int lectura = voltaje;
Sistem.out.println(lectura);
```

Respuesta: No compila
El error está en int lectura = voltaje; . voltaje es de tipo double, lectura es de tipo int. Java no permite convertir un double a int, porque se pierden decimales. Se necesitaría hacer un casting.

9. R2 c) ¿Compila, truena o funciona?

```
Integer reserva = 4;
int nueva = reserva + 3;
Sistem.out.println(nueva);
```

Respuesta: Funciona ✓
Reserva es de tipo integer, que es una clase envolvente de int. Java automaticamente convierte el valor de Integer a int y efectua la suma.
4+3 = 7. Imprime 7.

10. R_3 · Modelado: base, reto y justificación.

```
class Banda {
    private int piezas; // atributo de instancia. Es mutable, porque su valor cambia.
    private final int capacidad; // atributo de instancia. es final, su valor lo da el constructor.
                                 // y no se puede cambiar.
    Private static final int LIMITE_SEGURIDAD = 12;
    // Constante de clase, es el mismo para todas las bandas. static lo comparte y final impide
                                                              // modificarlo.
    Public Banda (int capacidad) {
        this.capacidad = capacidad; // Banda (int) capacidad) : es el constructor
        ths.piezas = 0              // que recibe la capacidad de la banda y establece
    }                               // el conteo.

    Public void agregarPieza () {   // Metodo void porque no devuelve un
        piezas ++;                  // resultado, aumenta de uno en uno el
    }                               // número de piezas.

    public int consultarPiezas() {  // Devuelve un int con la cantidad
        return piezas;              // actual de pieias.
    }
}
```

Reto: la capacidad debe ser final de instancia porque pertenece a cada banda y puede ser distinta entre objetos. En cambio, el límite de seguridad debe ser static final porque es una constante compartida.

11. R4 · Código con repetición

Un equipo parte con carga unidades. En cada paso suma aporte. Devuelve cuántos pasos necesita alcanzar o superar meta usando while. Si ya llegó, devuelve 0. Carga y meta estan entre 0 y 10000; aporte está entre 1 y 100. No uses formula de división.

```
Static int pasosCarga (int carga, int aporte, int meta ) {
 int pasos = 0;
 while (carga < meta) {
   carga += aporte;
   pasos ++;
}
 return pasos;
}
```

El metodo recibe la carga inicial, el aporte que se suma en cada caso y la meta que quiere alcanzar. La variable pasos comienza en 0.
Mientras carga sea menor que meta, el ciclo suma aporte a carga e incrementa el contador. Cuando la carga alcanza o supera la meta el ciclo termina y devuelve la cantidad de pasos realizados.

Ejemplos

Argumentos	Seguimiento de carga	Retorno	
5, 4, 16	5 -> 9 -> 13 -> 17	3	=> Necesita 3 pasos.
16, 4, 16	Ya alcanzó la meta	0	=> La carga inicial ya es la meta
20, 4, 16	Ya alcanzó la meta	0	=> La carga inicial ya supera la meta.

12. RS Codigo con Seleccion

Decide si un banco de pruebas puede arrancar. Recibes el porcentaje de bateria y si la tapa está cerrada. Aplica la primera regla que corresponda, en orden de la tabla. Usa if / else if / else, con && y ||. No imprimas ni leas del teclado.

Regla	Texto devuelto
Bateria menor que 0 o mayor que 100	"Invalida"
Tapa cerrada y bateria al menos 60	"Normal"
Tapa cerrada y bateria al menos 20	"Ahorro"
Cualquier otro caso	"Bloqueado"

```
static String permiso (int bateria, boolen tapaCerrada) {
  if (bateria < 0 || bateria > 100) {
    return "INVALIDA";
  } else if (tapaCerrada && bateria >= 60) {
    return "NORMAL";
  } else if (tapaCerrada && bateria >= 20) {
    return "AHORRO";
  } else {
    return "BLOQUEADO";
  }
}
```

Primero verifico que el porcentaje de bateria este dentro del rango permitido, de 0 a 100. Si no es asi, el método devuelve "INVALIDA". Despues compruebo si la tapa esta cerrada y la bateria es de al menos 60; si ambas condiciones se cumplen, devuelven "NORMAL". Sino, reviso si la tapa esta cerrada y la bateria es de al menos 20, en este caso devuelve "AHORRO". Si ninguna de las condiciones se cumple, devuelve "BLOQUEADO".

Hernandez Nieves
Emily.

Comprobación de las reglas:

Batería	Tapa cerrada	Resultado	¿Porqué?
-1	true	INVALIDA	Batería menor que 0.
101	true	INVALIDA	Batería mayor que 100.
60	true	NORMAL	La tapa está cerrada y batería es al menos 60.
20	true	AHORRO	Batería de al menos 20 y tapa cerrada
80	false	BLOQUEADO	La tapa esta abierta.
10	true	BLOQUEADO	La batería no alcanza 20.